Inteligență Artificială

Rețele neurale convoluționale

Slides: Mihai Trăscău, Mihai Nan

- Convoluții
- Straturi
- Pooling
- Rețele neurale convoluționale

- Pentru cazul continuu, produsul de convoluție dintre două funcții f și g este definit ca:

$$(f * g)(t) = \int_{-\infty}^{+\infty} f(\tau)g(t-\tau)d\tau = \int_{-\infty}^{\infty} f(t-\tau)g(\tau)d\tau$$

- Pentru cazul discret, integrala este înlocuită cu sumă:

$$(f * g)(n) = \sum_{m=-\infty}^{\infty} f(m)g(n-m) = \sum_{m=-\infty}^{\infty} f(n-m)g(m)$$

- O imagine **RGB** poate fi considerată o funcție ($f : \mathbb{R}^2 \rightarrow \mathbb{R}^3$), unde $f(x, y) = (red_value, green_value, blue_value)^T$
- Ca în cazul oricărei funcții, putem aplica operatori asupra imaginilor



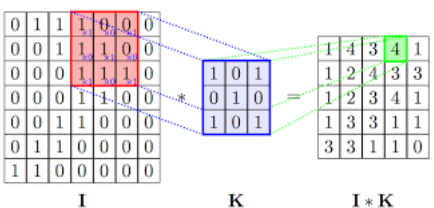
$g(x,y) = f(x,y) + 20$ $\quad$ $g(x,y) = f(-x,y)$

- Operatorul de convoluție stă la baza aplicării operațiilor liniare de filtrare a imaginilor aplicate în domeniul spațial (în planul imagine prin manipularea directă a pixelilor din imagine).

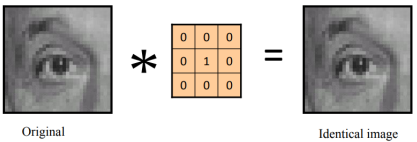
- Operația de convoluție implică folosirea unui nucleu de convoluție (K - de dimensiune $w \times w$, unde $w = 2k + 1$) care se aplică peste imaginea sursă (I) în conformitate cu:

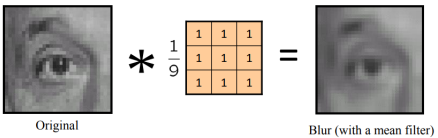
$$O = I * K$$

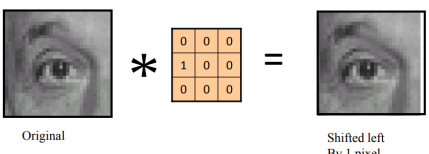
$$O(i,j) = \sum_{u=0}^{w-1} \sum_{v=0}^{w-1} I(i+u-k, j+v-k) \cdot K(u,v)$$

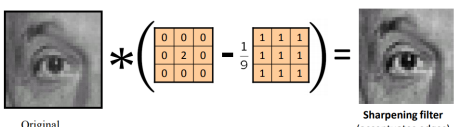


- Operația de convoluție este **asociativă** și **comutativă**





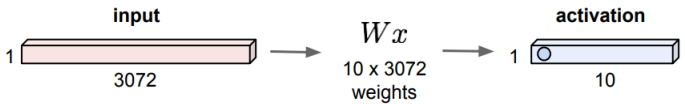




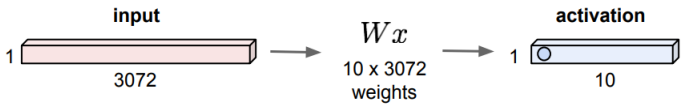
- Pornim de la o imagine având dimensiunea $32 \times 32 \times 3$, dorim să îi atribuim o clasă din 10 disponibile.

- Pornim de la o imagine având dimensiunea $32 \times 32 \times 3$, dorim să îi atribuim o clasă din 10 disponibile.
  1. Liniarizăm imaginea, transformând-o într-un vector, având dimensiunea 3072×1.

- Pornim de la o imagine având dimensiunea $32 \times 32 \times 3$, dorim să îi atribuim o clasă din 10 disponibile.
  1. Liniarizăm imaginea, transformând-o într-un vector, având dimensiunea 3072×1.
  2. Trecem vectorul printr-un strat complet conectat (Fully Connected Layer).



- Pornim de la o imagine având dimensiunea $32 \times 32 \times 3$, dorim să îi atribuim o clasă din 10 disponibile.
  1. Liniarizăm imaginea, transformând-o într-un vector, având dimensiunea 3072×1.
  2. Trecem vectorul printr-un strat complet conectat (Fully Connected Layer).



3. Din punct de vedere matematic, aplicăm următoarea operație:

$$\begin{bmatrix} w_{1,1} & w_{1,2} & \dots & w_{1,3072} \\ \vdots & \vdots & \vdots & \vdots \\ w_{10,1} & w_{10,2} & \dots & w_{10,3072} \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{3072} \end{bmatrix} + \begin{bmatrix} b_1 \\ \vdots \\ b_{10} \end{bmatrix} = \begin{bmatrix} y_1 \\ \vdots \\ y_{10} \end{bmatrix} \tag{1}$$

$$\begin{bmatrix} w_{1,1} & w_{1,2} & \dots & w_{1,3072} \\ \vdots & \vdots & \vdots & \vdots \\ w_{10,1} & w_{10,2} & \dots & w_{10,3072} \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{3072} \end{bmatrix} + \begin{bmatrix} b_1 \\ \vdots \\ b_{10} \end{bmatrix} = \begin{bmatrix} y_1 \\ \vdots \\ y_{10} \end{bmatrix}$$

4. În formă restrânsă, ecuația de mai sus se poate rescrie ca:

$$H(X) = (W \cdot x) + b^T$$

$$H(X) = (W^T \cdot x) + b$$

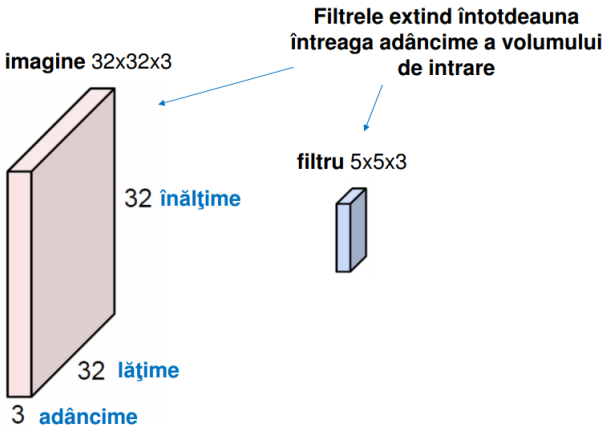
$$\begin{bmatrix} w_{1,1} & w_{1,2} & \dots & w_{1,3072} \\ \vdots & \vdots & \vdots & \vdots \\ w_{10,1} & w_{10,2} & \dots & w_{10,3072} \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{3072} \end{bmatrix} + \begin{bmatrix} b_1 \\ \vdots \\ b_{10} \end{bmatrix} = \begin{bmatrix} y_1 \\ \vdots \\ y_{10} \end{bmatrix}$$

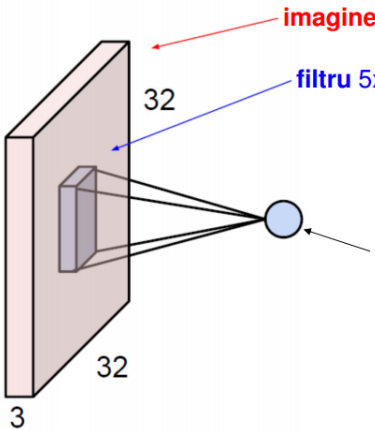
4. În formă restrânsă, ecuația de mai sus se poate rescrie ca:

$$H(X) = (W \cdot x) + b^T$$
$$H(X) = (W^T \cdot x) + b$$

- Această metodă are ca dezavantaj major faptul că se pierde dispunerea spațială a pixelilor din imagine

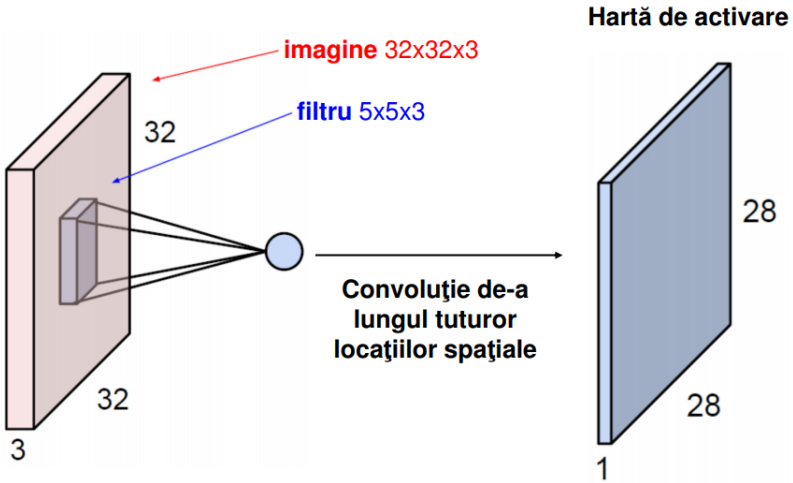
- Soluție: putem păstra imaginea în forma inițială și să aplicăm un strat convoluțional asupra ei.





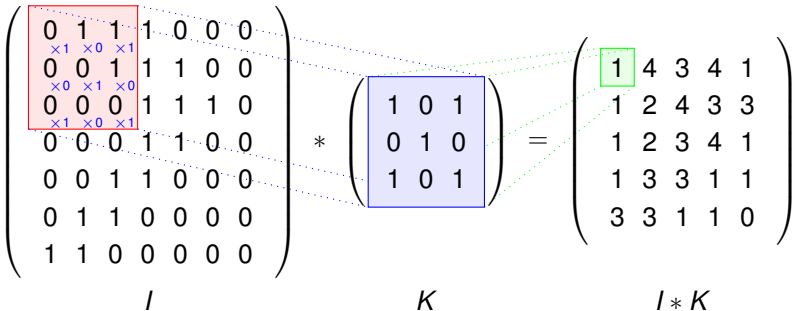
1 număr:
Rezultatul produsului scalar dintre filtru şi o regiune de interes 5x5x3 din imagine (ex: 5*5*3 = produs scalar de dimensiune 75 + bias)

$$\theta^T x + b$$

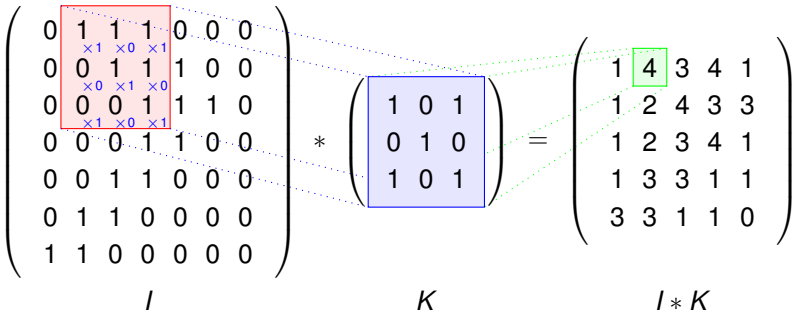

Hartă de activare
imagine 32x32x3
filtru 5x5x3
32
32
3
Convoluţie de-a lungul tuturor locaţiilor spaţiale
28
28
1

- Întrebare: De ce dacă pornim de la o imagine de forma $32 \times 32 \times 3$ și aplicăm un filtru de forma $5 \times 5 \times 3$ vom ajunge la o hartă de activare având dimensiunea $28 \times 28 \times 1$?

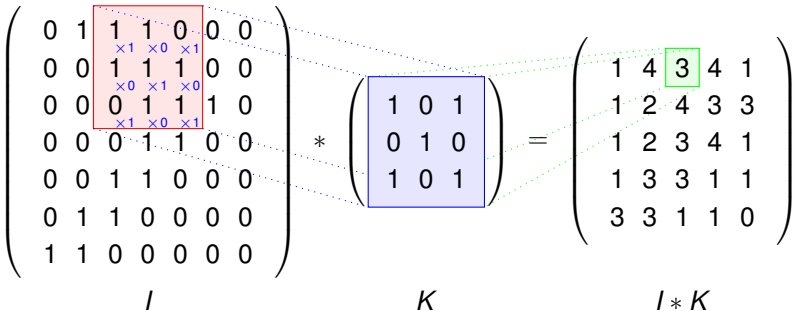
- Întrebare: De ce dacă pornim de la o imagine de forma $32 \times 32 \times 3$ și aplicăm un filtru de forma $5 \times 5 \times 3$ vom ajunge la o hartă de activare având dimensiunea $28 \times 28 \times 1$?



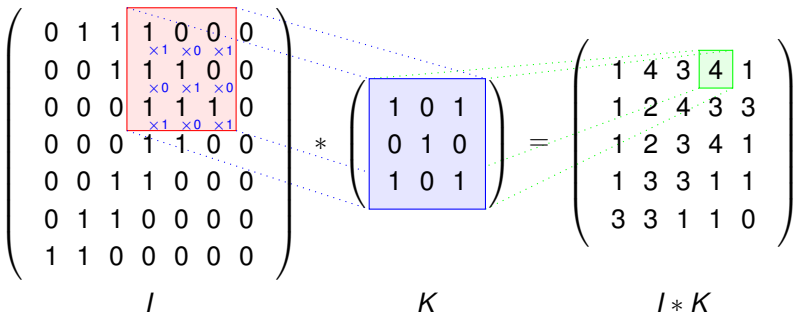
- Întrebare: De ce dacă pornim de la o imagine de forma $32 \times 32 \times 3$ și aplicăm un filtru de forma $5 \times 5 \times 3$ vom ajunge la o hartă de activare având dimensiunea $28 \times 28 \times 1$?



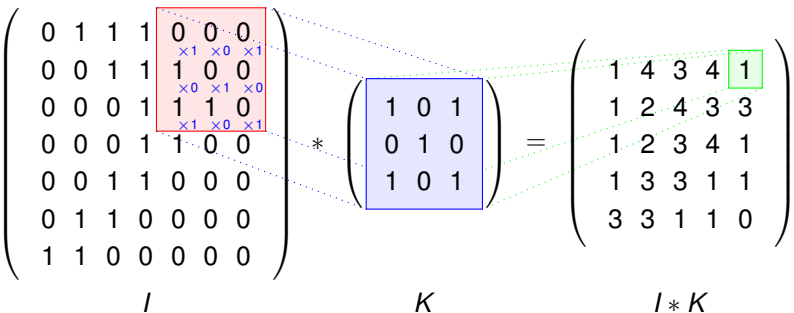
- Întrebare: De ce dacă pornim de la o imagine de forma $32 \times 32 \times 3$ și aplicăm un filtru de forma $5 \times 5 \times 3$ vom ajunge la o hartă de activare având dimensiunea $28 \times 28 \times 1$?



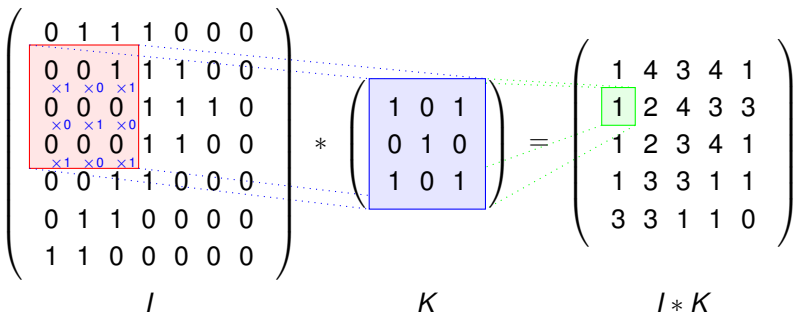
- Întrebare: De ce dacă pornim de la o imagine de forma $32 \times 32 \times 3$ și aplicăm un filtru de forma $5 \times 5 \times 3$ vom ajunge la o hartă de activare având dimensiunea $28 \times 28 \times 1$?



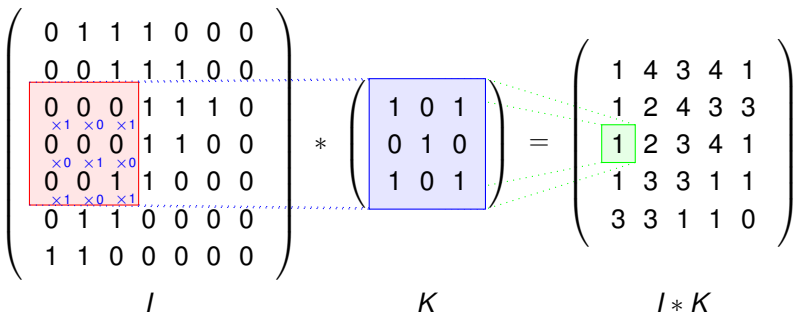
- Întrebare: De ce dacă pornim de la o imagine de forma $32 \times 32 \times 3$ și aplicăm un filtru de forma $5 \times 5 \times 3$ vom ajunge la o hartă de activare având dimensiunea $28 \times 28 \times 1$?



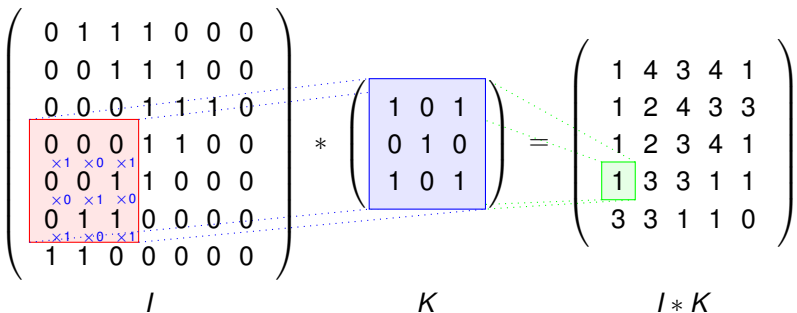
- Întrebare: De ce dacă pornim de la o imagine de forma $32 \times 32 \times 3$ și aplicăm un filtru de forma $5 \times 5 \times 3$ vom ajunge la o hartă de activare având dimensiunea $28 \times 28 \times 1$?



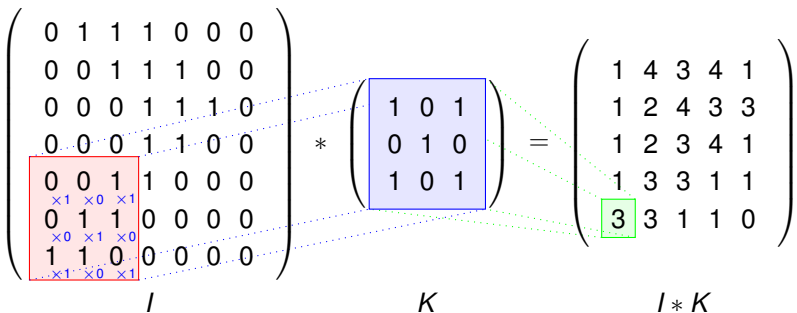
- Întrebare: De ce dacă pornim de la o imagine de forma $32 \times 32 \times 3$ și aplicăm un filtru de forma $5 \times 5 \times 3$ vom ajunge la o hartă de activare având dimensiunea $28 \times 28 \times 1$?



- Întrebare: De ce dacă pornim de la o imagine de forma $32 \times 32 \times 3$ și aplicăm un filtru de forma $5 \times 5 \times 3$ vom ajunge la o hartă de activare având dimensiunea $28 \times 28 \times 1$?



- Întrebare: De ce dacă pornim de la o imagine de forma $32 \times 32 \times 3$ și aplicăm un filtru de forma $5 \times 5 \times 3$ vom ajunge la o hartă de activare având dimensiunea $28 \times 28 \times 1$?

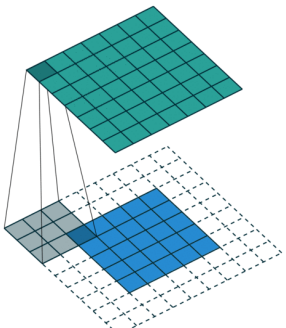


- Întrebare: De ce dacă pornim de la o imagine de forma $32 \times 32 \times 3$ și aplicăm un filtru de forma $5 \times 5 \times 3$ vom ajunge la o hartă de activare având dimensiunea $28 \times 28 \times 1$?
- Răspuns: Dacă pornim de la o imagine de forma $Height \times Width$ și aplicăm un strat convoluțional cu un filtru de forma $K \times K \times 3$ vom ajunge la o hartă de activare cu dimensiunea $[Height - (K - 1)] \times [Width - (K - 1)]$

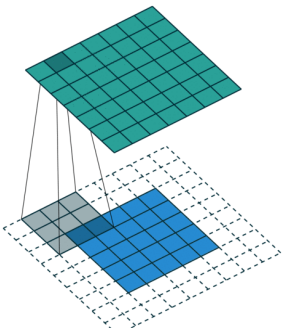
- Întrebare: Cum putem păstra dimensiunea inițială a imaginii și pentru harta de activare?

- Întrebare: Cum putem păstra dimensiunea inițială a imaginii și pentru harta de activare?
- Răspuns: Vom folosi noțiunea de padding (bordare)

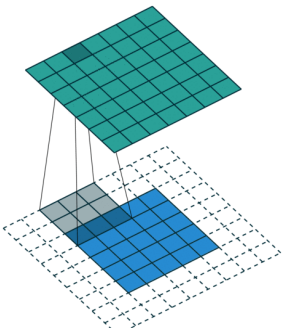
- Întrebare: Cum putem păstra dimensiunea inițială a imaginii și pentru harta de activare?
- Răspuns: Vom folosi noțiunea de padding (bordare)



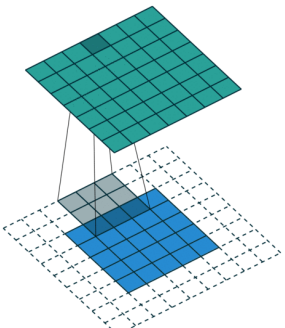
- Întrebare: Cum putem păstra dimensiunea inițială a imaginii și pentru harta de activare?
- Răspuns: Vom folosi noțiunea de padding (bordare)



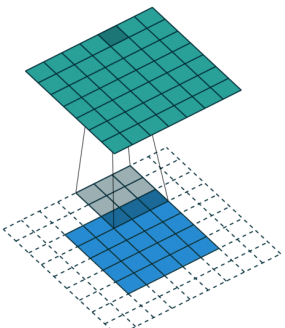
- Întrebare: Cum putem păstra dimensiunea inițială a imaginii și pentru harta de activare?
- Răspuns: Vom folosi noțiunea de padding (bordare)



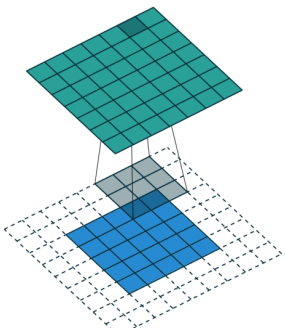
- Întrebare: Cum putem păstra dimensiunea inițială a imaginii și pentru harta de activare?
- Răspuns: Vom folosi noțiunea de padding (bordare)



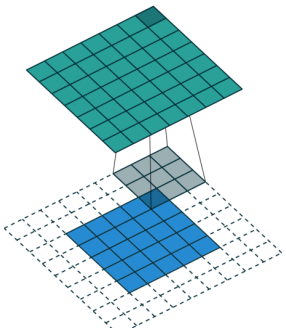
- Întrebare: Cum putem păstra dimensiunea inițială a imaginii și pentru harta de activare?
- Răspuns: Vom folosi noțiunea de padding (bordare)



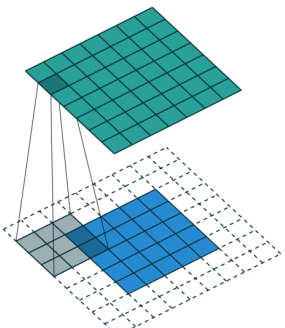
- Întrebare: Cum putem păstra dimensiunea inițială a imaginii și pentru harta de activare?
- Răspuns: Vom folosi noțiunea de padding (bordare)



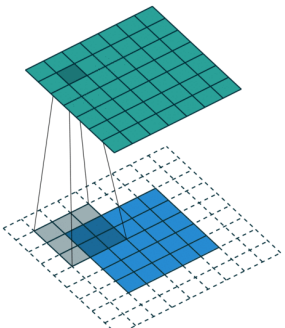
- Întrebare: Cum putem păstra dimensiunea inițială a imaginii și pentru harta de activare?
- Răspuns: Vom folosi noțiunea de padding (bordare)



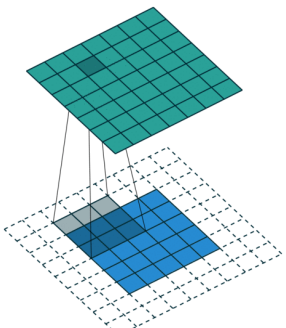
- Întrebare: Cum putem păstra dimensiunea inițială a imaginii și pentru harta de activare?
- Răspuns: Vom folosi noțiunea de padding (bordare)



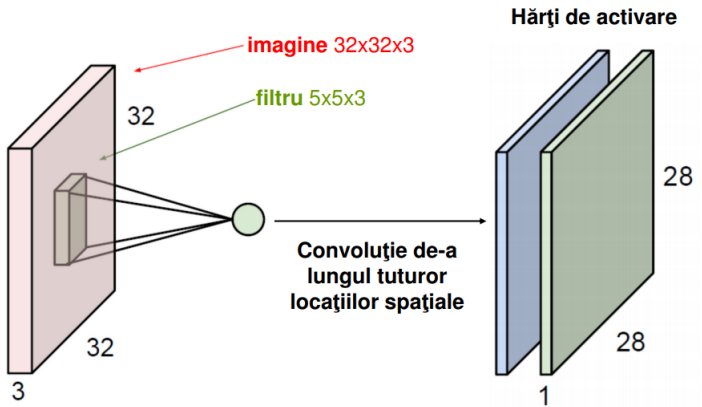
- Întrebare: Cum putem păstra dimensiunea inițială a imaginii și pentru harta de activare?
- Răspuns: Vom folosi noțiunea de padding (bordare)



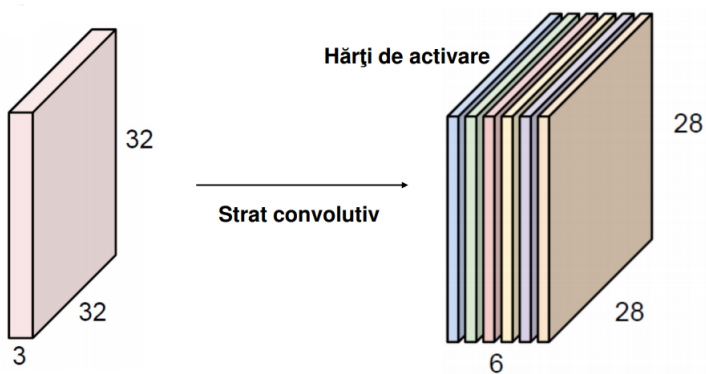
- Întrebare: Cum putem păstra dimensiunea inițială a imaginii și pentru harta de activare?
- Răspuns: Vom folosi noțiunea de padding (bordare)

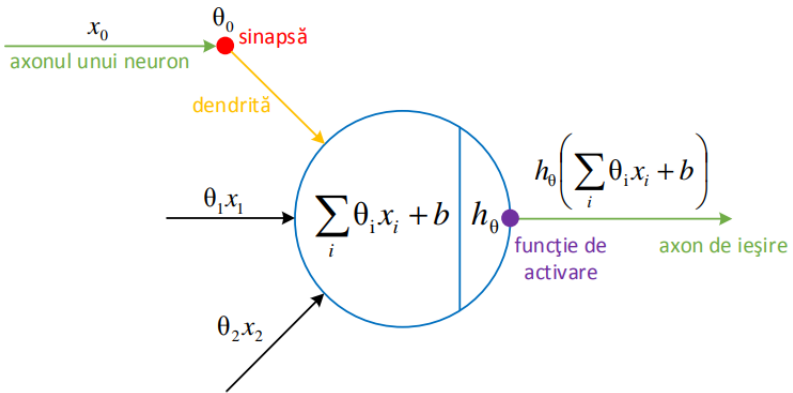


- Putem aplica mai multe filtre pentru o imagine, urmând să obținem o hartă de activare pentru fiecare filtru.
- Exemplu: Aplicarea unui filtru pentru verde



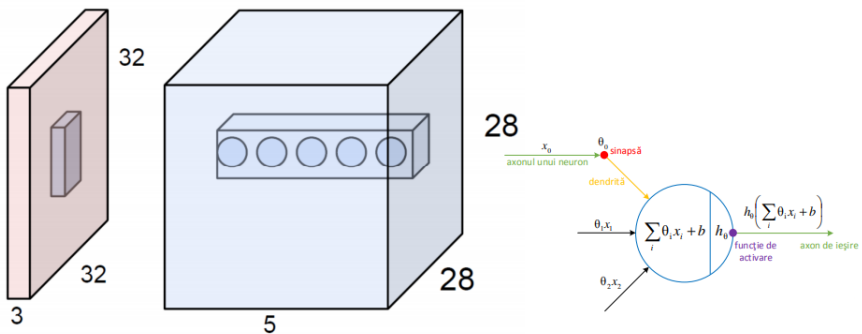
- Dacă vrem să aplicăm 6 filtre cu un kernel cu dimensiunea $5 \times 5 \times 3$, vom obține o *„imagine”* cu dimensiunea $28 \times 28 \times 6$.



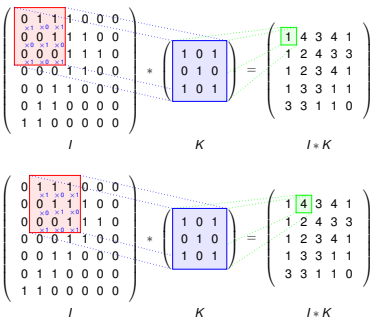


- Fiecare neuron este conectat la o mică regiune de interes din volumul de intrare.
- Toți neuronii împart același set de parametri Θ.

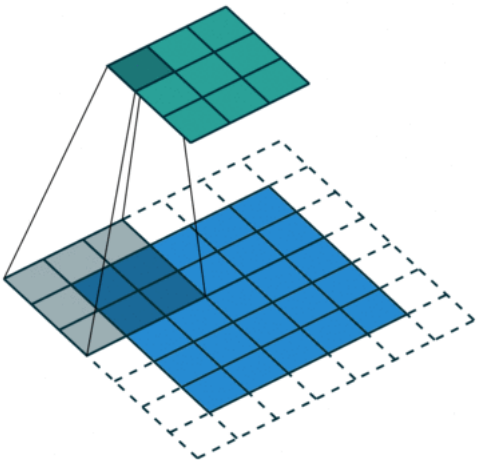
- **Exemplu:** Având 5 filtre, stratul convoluțional este alcătuit din neuroni ordonați într-un grid $3D$ de dimensiune $28 \times 28 \times 5$. Astfel, vor exista 5 neuroni diferiți, toți analizând aceeași regiune din volumul de intrare.



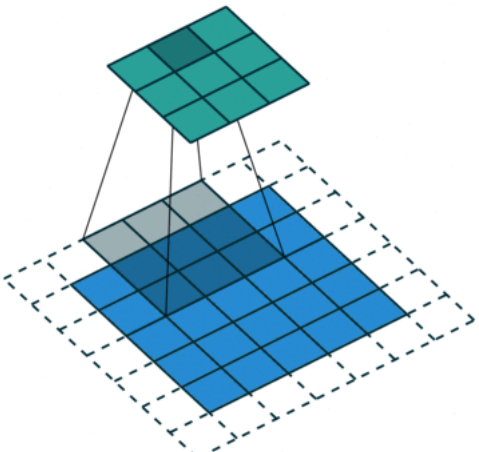
- Stride (deplasarea) = numărul de pixeli peste care sare filtrul la generarea hărții de activare.
- Exemplu: stride = 1



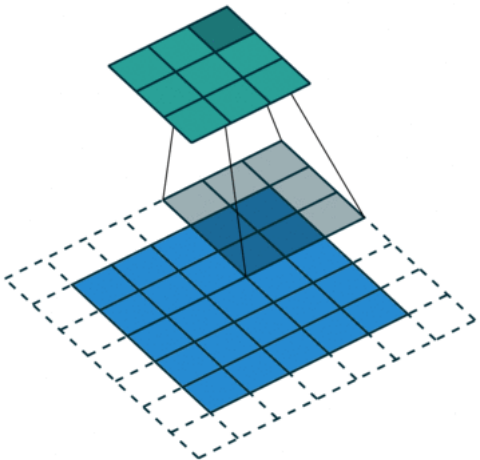
- Exemplu: padding = 1, stride = 2



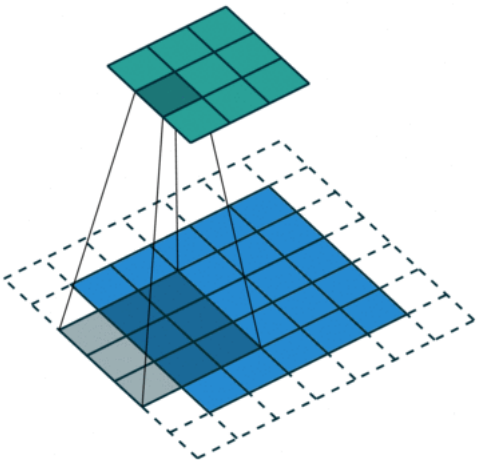
- Exemplu: padding = 1, stride = 2



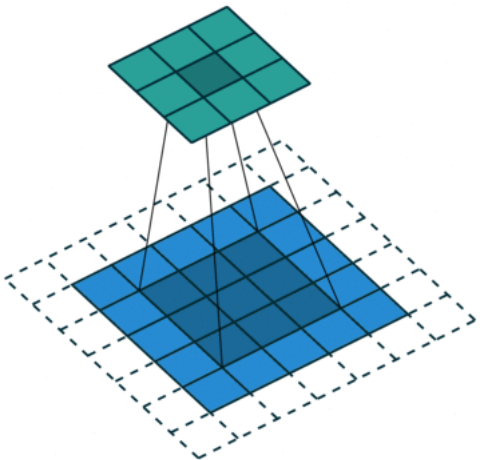
- Exemplu: padding = 1, stride = 2



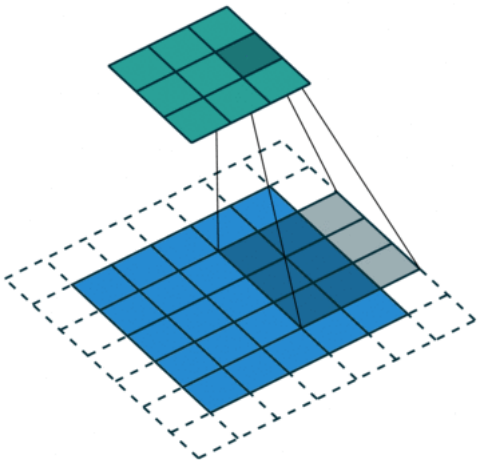
- Exemplu: padding = 1, stride = 2



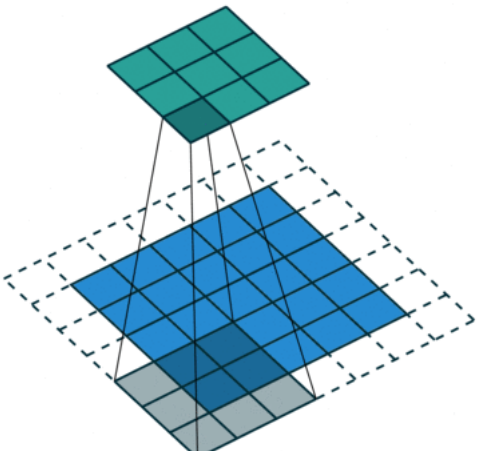
- Exemplu: padding = 1, stride = 2



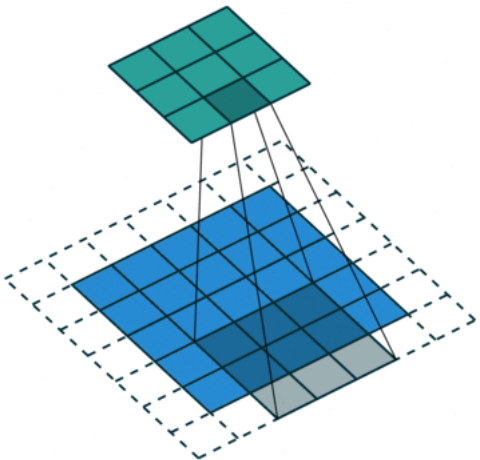
- Exemplu: padding = 1, stride = 2



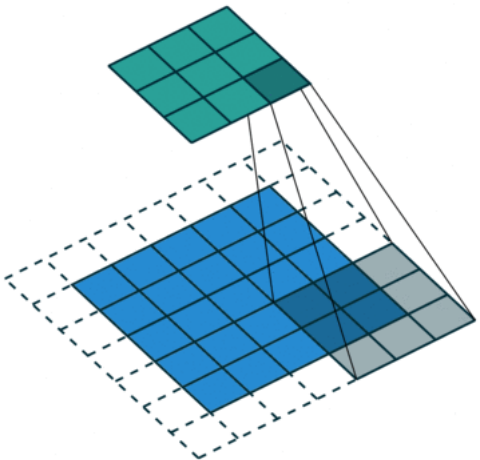
- Exemplu: padding = 1, stride = 2



- Exemplu: padding = 1, stride = 2



- Exemplu: padding = 1, stride = 2



- Întrebare: Cum influențează valoarea deplasării dimensiunea hărții de activare?

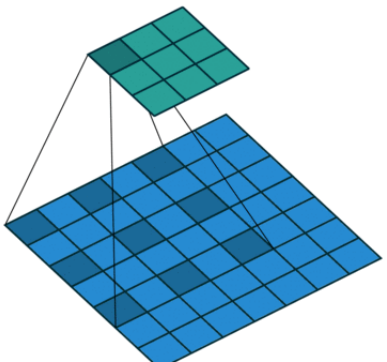
- Întrebare: Cum influențează valoarea deplasării dimensiunea hărții de activare?
- Răspuns:
  - stride = 1 $\Rightarrow$ hartă de activare cu aceeași dimensiune ca imaginea inițială;
  - stride $>$ 1 $\Rightarrow$ hartă de activare cu dimensiune mai mică.

- Întrebare: Cum influențează valoarea deplasării dimensiunea hărții de activare?
- Răspuns:
  - stride = 1 $\Rightarrow$ hartă de activare cu aceeași dimensiune ca imaginea inițială;
  - stride $> 1 \Rightarrow$ hartă de activare cu dimensiune mai mică.

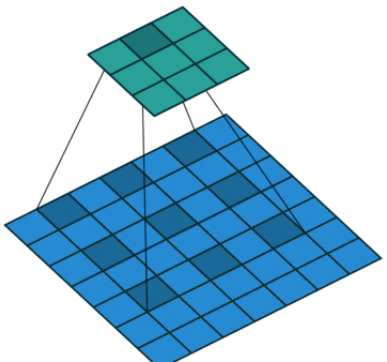
$$H_{out} = \left[\frac{H_{in} + 2 * padding - (kernel_size - 1) - 1}{stride} + 1 \right]$$

$$W_{out} = \left[\frac{W_{in} + 2 * padding - (kernel_size - 1) - 1}{stride} + 1 \right]$$

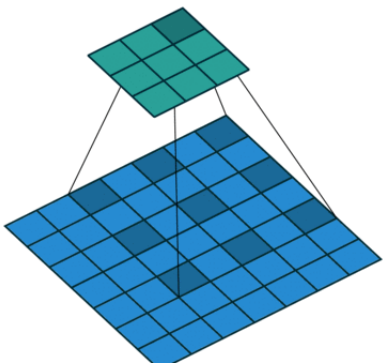
- Dilation (dilatare) reprezintă distanța dintre elementele kernelului.
- Exemplu: dilation = 2



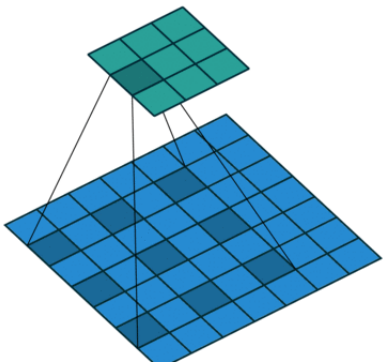
- Dilation (dilatare) reprezintă distanța dintre elementele kernelului.
- Exemplu: dilation = 2



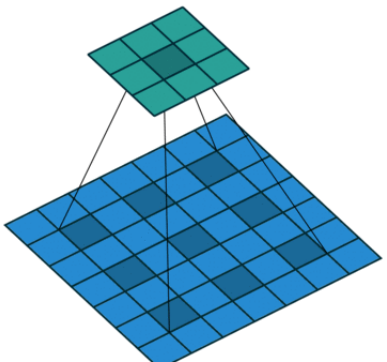
- Dilation (dilatare) reprezintă distanța dintre elementele kernelului.
- Exemplu: dilation = 2



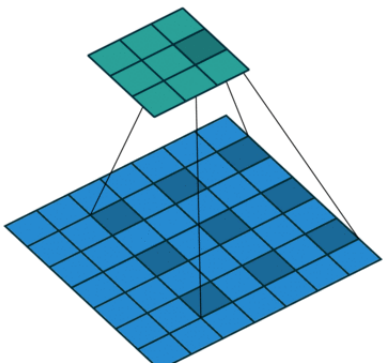
- Dilation (dilatare) reprezintă distanța dintre elementele kernelului.
- Exemplu: dilation = 2



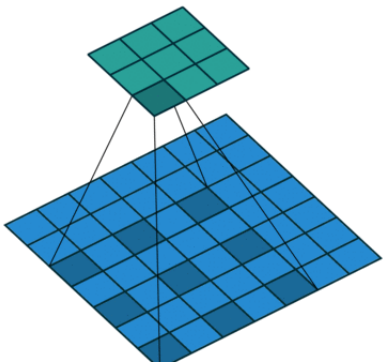
- Dilation (dilatare) reprezintă distanța dintre elementele kernelului.
- Exemplu: dilation = 2



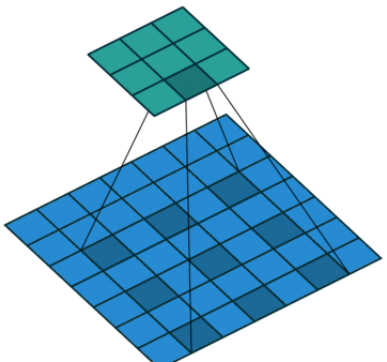
- Dilation (dilatare) reprezintă distanța dintre elementele kernelului.
- Exemplu: dilation = 2



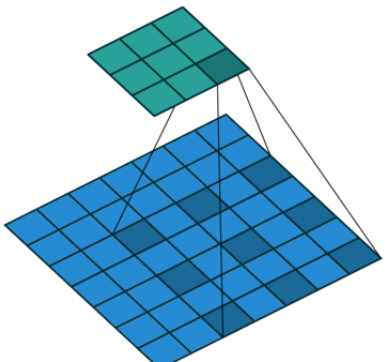
- Dilation (dilatare) reprezintă distanța dintre elementele kernelului.
- Exemplu: dilation = 2



- Dilation (dilatare) reprezintă distanța dintre elementele kernelului.
- Exemplu: dilation = 2



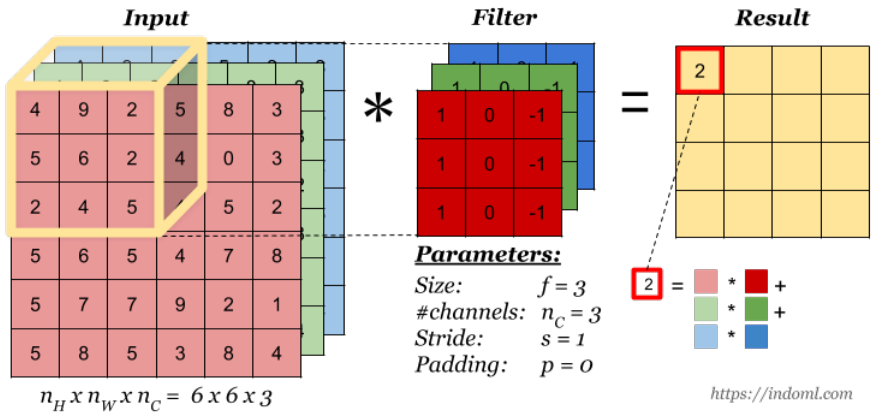
- Dilation (dilatare) reprezintă distanța dintre elementele kernelului.
- Exemplu: dilation = 2



- Întrebare: Cum influențează valoarea dilatării dimensiunea hărții de activare?
- Răspuns:

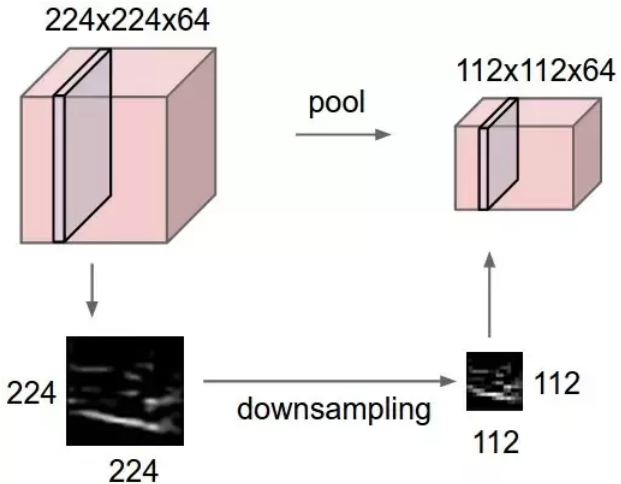
$$H_{out} = \left[\frac{H_{in} + 2 * padding - dilation * (kernel_size - 1) - 1}{stride} + 1 \right]$$

$$W_{out} = \left[\frac{W_{in} + 2 * padding - dilation * (kernel_size - 1) - 1}{stride} + 1 \right]$$

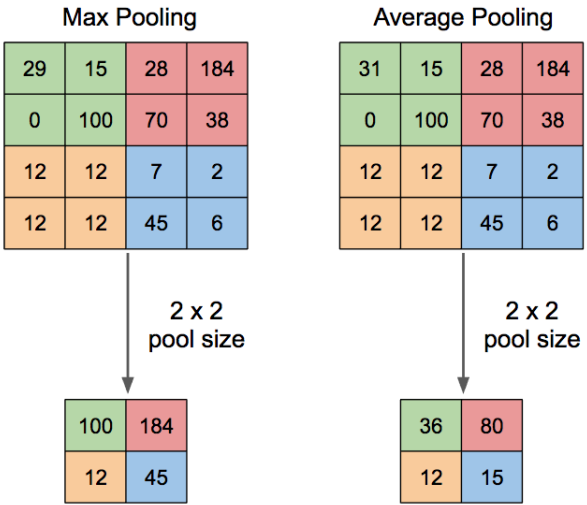


- Numărul din canale din filtru trebuie să coincidă cu numărul de canale din imagine.

- Operația de pooling modifică volumul într-o reprezentare mai mică și mai flexibilă;
- Operează independent de-a lungul fiecărei hărți de activare.

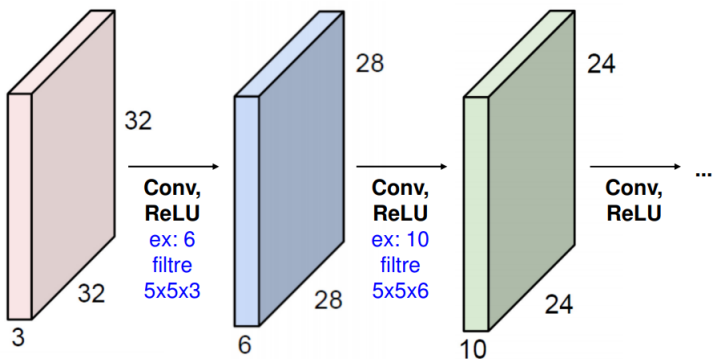


- Există mai multe moduri în care operația de pooling poate fi implementată.



Rețele neurale convoluționale

- O rețea neurală convoluțională este o secvență de straturi de convoluție intercalate cu funcții de activare (ex. ReLU, Sigmoid).
- În practică, utilizăm și straturi de pooling deoarece lucrăm cu imagini de dimensiuni mari relativ la numărul de parametri ai straturilor.



Rețele neurale convoluționale : Propagare forward

- Cross-corelation

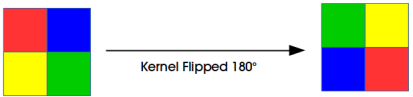
$$(I \otimes K)_{ij} = \sum_{m=0}^{k_1-1} \sum_{n=0}^{k_2-1} I(i+m, j+n) K(m,n)$$

- Convoluție

$$\begin{aligned}(I * K)_{ij} &= \sum_{m=0}^{k_1-1} \sum_{n=0}^{k_2-1} I(i-m, j-n) K(m,n) \\ &= \sum_{m=0}^{k_1-1} \sum_{n=0}^{k_2-1} I(i+m, j+n) K(-m,-n)\end{aligned}$$

- Convoluția o să coincidă cu cross-correlation dacă vom oglindi kernelul.

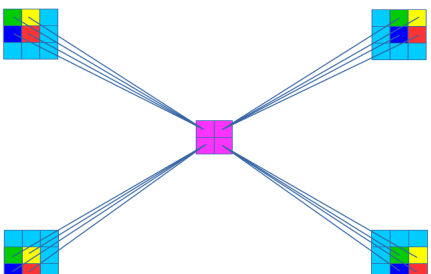
Rețele neurale convoluționale : Propagare forward



$$x_{i,j}^{l} = \text{rot}_{180^\circ}\left\{w_{m,n}^{l}\right\} * o_{i,j}^{l-1} + b_{i,j}^{l}$$

$$x_{i,j}^{l} = \sum_{m}\sum_{n} w_{m,n}^{l} o_{i+m,j+n}^{l-1} + b_{i,j}^{l}$$

$$o_{i,j}^{l} = f(x_{i,j}^{l})$$



Rețele neurale convoluționale : Funcții de eroare

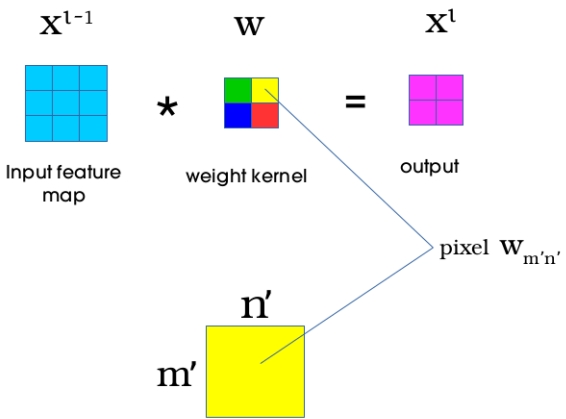
- Dacă folosim rețeaua pentru clasificare multi-clasă putem folosi categorical cross-entropy

$$CE(p, q) = -\sum_{j=1}^{D} y_j \log(f(x_j, \theta))$$

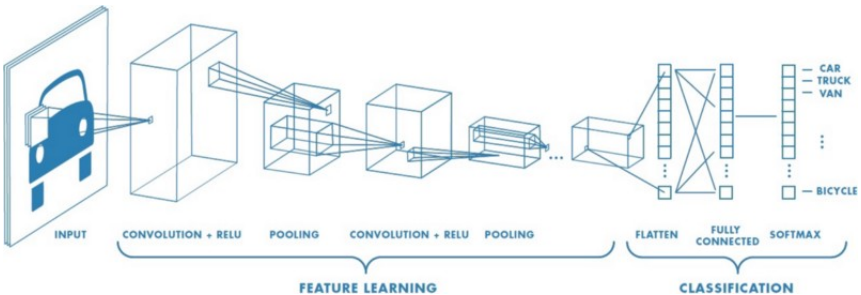
- y_j - target-ul;
- $f(x_j, \theta)$ - predicția rețelei;
- D - dimensiunea batch-ului;
- j - un tensor (exemplu) din batch.

Rețele neurale convoluționale : Backpropagation

- Vom calcula $\frac{\partial E}{\partial w^l_{m',n'}}$ care poate fi interpretată ca măsurarea modului în care schimbarea valorii ponderii unui pixel din kernel ($w_{m',n'}$) afectează rezultatul funcției de eroare E.



Rețele neurale convoluționale : Aplicații



Rețele neurale convoluționale : Cod Pytorch

```python
class ConvNet(nn.Module):
    def __init__(self, in_channel=3, num_classes=10):
        super(ConvNet, self).__init__()
        self.conv1 = nn.Conv2d(in_channel, 4, kernel_size=5, stride=1, padding=1)
        self.conv2 = nn.Conv2d(4, 8, kernel_size=5, stride=1, padding=1)
        self.conv3 = nn.Conv2d(8, 16, kernel_size=3, padding=1)
        self.conv4 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.conv5 = nn.Conv2d(32, 16, kernel_size=3)
        self.conv6 = nn.Conv2d(16, num_classes, kernel_size=3, stride=1, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2)
        self.relu = nn.ReLU()
        self.fc1 = nn.Linear(10*11*11, 10*3*3)
        self.fc2 = nn.Linear(10*3*3, 10*3)
        self.fc3 = nn.Linear(10*3, num_classes)

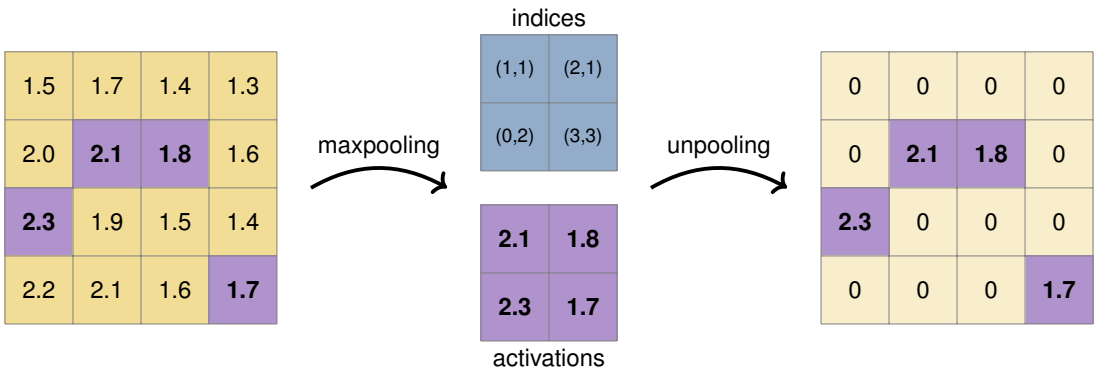
    def forward(self, x):
        out = self.relu(self.conv1(x))
        out = self.pool(out)
        out = self.relu(self.conv2(out))

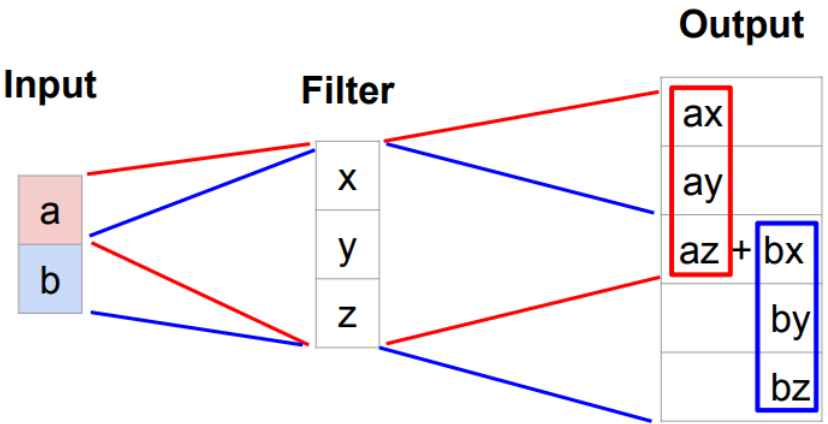
        out = self.relu(self.conv3(out))
        out = self.relu(self.conv4(out))

        out = self.relu(self.conv5(out))
        out = self.relu(self.conv6(out))
        out = out.view(out.size(0), -1)
        out = self.fc1(out)
        out = self.fc2(out)
        out = self.fc3(out)
        return out
```

Rețele neurale convoluționale : Max Unpooling

- Max Unpooling este operația inversă (directă) a celei de pooling




Output
Input
Filter
a
b
x
y
z
ax
ay
az + bx
by
bz

Rețele neurale convoluționale : Convoluție transpusă

- Convoluție - $kernel_size = 3, padding = 1, stride = 1$ $(\vec{x} * \vec{a} = X\vec{a})$

$$\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & x & y & z & 0 & 0 \\ 0 & 0 & x & y & z & 0 \\ 0 & 0 & 0 & x & y & z \end{bmatrix} \cdot \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ ax + by + cz \\ bx + cy + dz \\ cx + dy \end{bmatrix}$$

- Convoluție transpusă - $stride = 1$ - $(x *^T \vec{a} = X^T \vec{a})$

$$\begin{bmatrix} x & 0 & 0 & 0 \\ y & x & 0 & 0 \\ z & y & x & 0 \\ 0 & z & y & x \\ 0 & 0 & z & y \\ 0 & 0 & 0 & z \end{bmatrix} \cdot \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} ax \\ ay + bx \\ az + by + cx \\ bz + cy + dx \\ cz + dy \\ dz \end{bmatrix}$$

- Când $stride = 1$ convoluția transpusă este doar o convoluție normală (cu reguli diferite de bordare).

Rețele neurale convoluționale : Convoluție transpusă

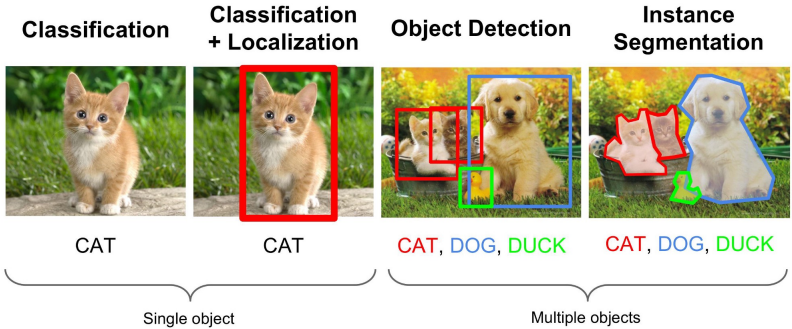
- Convoluție - $kernel_size = 3, padding = 1, stride = 2$ $(\vec{x} * \vec{a} = X\vec{a})$

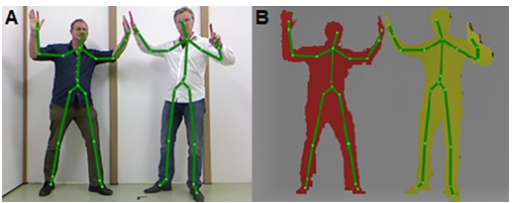
$$\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & 0 & x & y & z & 0 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ bx + cy + dz \end{bmatrix}$$

- Convoluție transpusă - $stride = 1$ - $(x *^T \vec{a} = X^T \vec{a})$

$$\begin{bmatrix} x & 0 \\ y & 0 \\ z & x \\ 0 & y \\ 0 & z \\ 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} ax \\ ay \\ az + bx \\ by \\ bz \\ 0 \end{bmatrix}$$

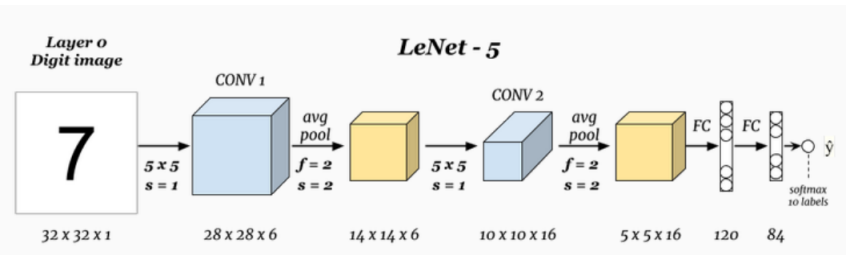
Rețele neurale convoluționale : Arhitecturi și Exemple





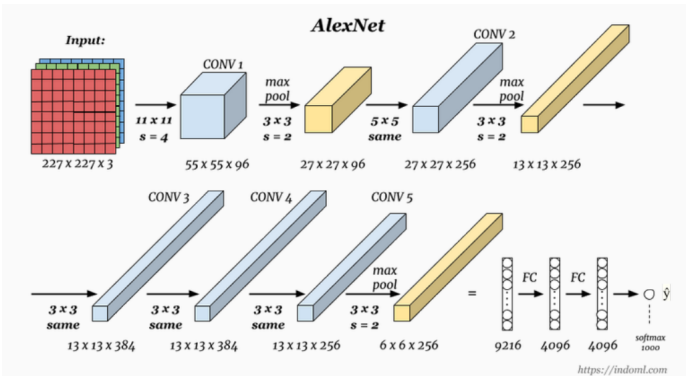
Rețeaua LeNet-5 – Y. Lecun, L. Bottou, Y. Bengio, P. Haffner (1998)

- Utilizată pentru recunoașterea unor imagini stil thumbnail cu forma digitalizată a unor cifre scrise de mână (setul de date MNIST).
- Număr parametrii: 60000



Rețeaua AlexNet – Alex Krizhevsky, Geoffrey Hinton, Ilya Sutskever (2012)

- Prima rețea înscrisă (și care a câștigat) în competiția **ImageNet-1K**
- Număr parametrii: 60 milioane



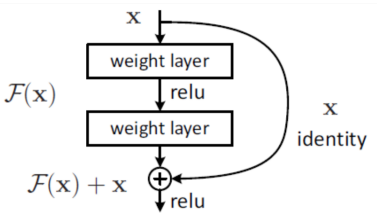
Rețele neurale convoluționale : Arhitecturi și Exemple

Rețeaua AlexNet – Alex Krizhevsky, Geoffrey Hinton, Ilya Sutskever (2012)

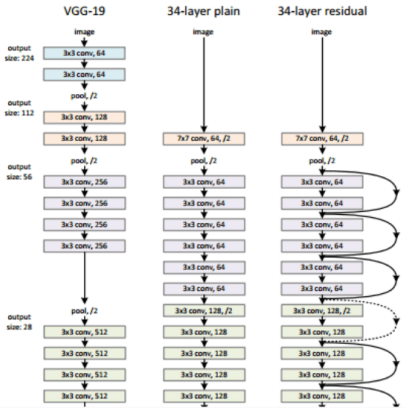
- Prima rețea înscrisă (și care a câștigat) în competiția **ImageNet-1K**
- Număr parametrii: 60 milioane
- Prima utilizare a funcției de activare ReLU
- Regularizare:
  - dropout: 0.5
  - regularizare pe ponderi: L2
- Optimizare: SGD + momentum, rata de învățare pornind de la 0.01 și ajustata *manual* când acuratețea stagna
- Utilizare **metodă de ansamblu**: utilizarea a 7 rețele antrenate în paralel pentru predicția finală pentru a beneficia de inițializări mai oportune ale ponderilor (scădere a procentului de erori top-5 de la 18.2% –> 15.2%)

Rețeaua ResNet – Kaiming He et al. (2015)

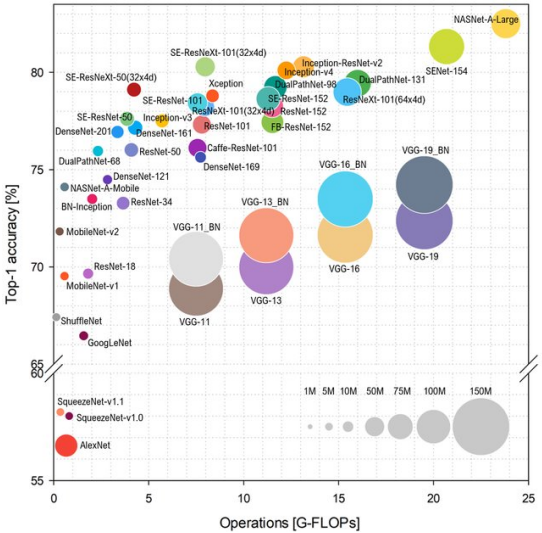
- Câștigător competiție ImageNet 2015: rata de eroare top-5 de 3.6% (mai mică decât performanța umană: 5.1%)



$$y = x + F(x) \Rightarrow \frac{\partial J}{\partial x} = \frac{\partial J}{\partial y} \cdot \frac{\partial y}{\partial x} = \frac{\partial J}{\partial y} \cdot (1 + F'(x))$$



Rețele neurale convoluționale : Arhitecturi și Exemple



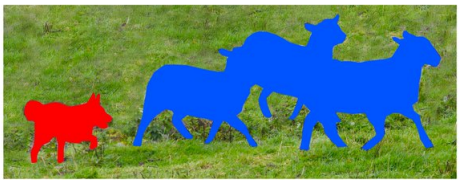
Sursă: Benchmark analysis of representative deep neural network architectures, 2018, S. Bianco et al.

Rețele neurale convoluționale : Arhitecturi și Exemple

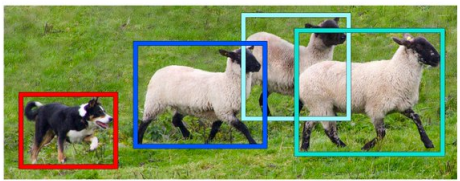
- Exemplu de arhitectură Fully Convolutional pentru **segmentare semantică**.



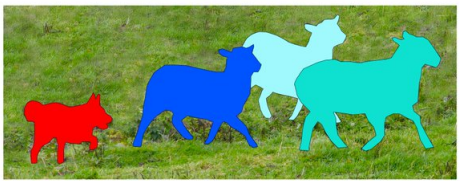
Image Recognition



Semantic Segmentation



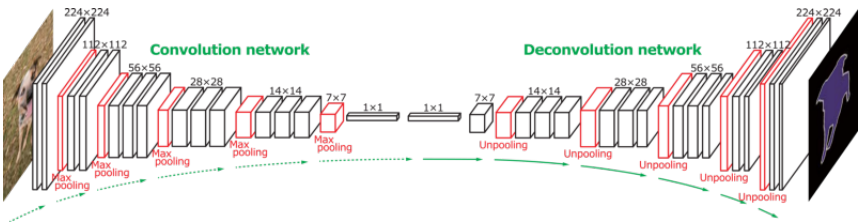
Object Detection



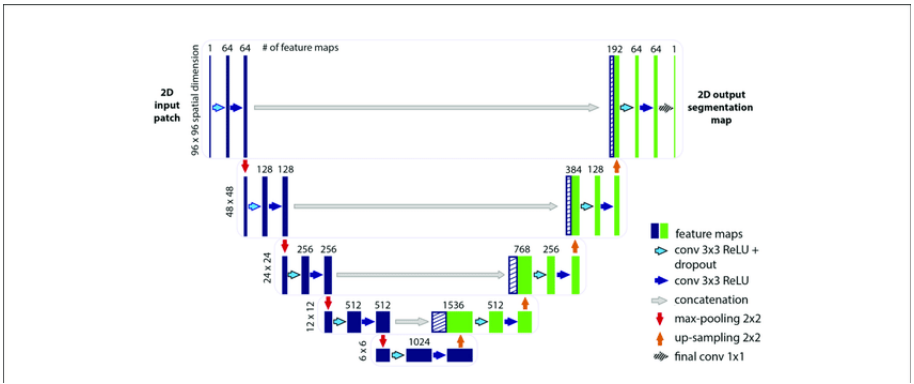
Instance Segmentation

Rețele neurale convoluționale : Arhitecturi și Exemple

- Exemplu de arhitectură Fully Convolutional pentru **segmentare semantică**.



Arhitecturi tip U-Net



Arhitectura U-Net. Sursa: A U-Net Deep Learning Framework for High Performance Vessel Segmentation in Patients With Cerebrovascular Disease, Livne et al., 2019, Frontiers in Neuroscience

Arhitectura YOLO – You only look once

- Familie de modele întinzându-se pe 8 iterații: v1 – 2016, v8 – 2024 (https://yolov8.com/)
- Folosite pentru **detecția de obiecte** și **segmentare**

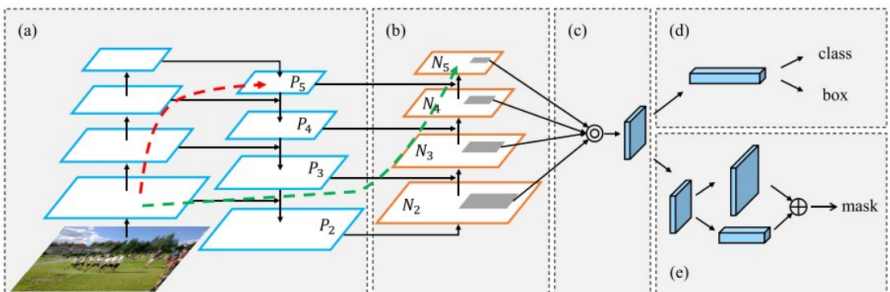


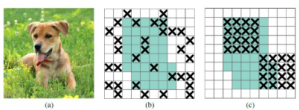
Figure 1. Illustration of our framework. (a) FPN backbone. (b) Bottom-up path augmentation. (c) Adaptive feature pooling. (d) Box branch. (e) Fully-connected fusion. Note that we omit channel dimension of feature maps in (a) and (b) for brevity.

Path Aggregation Network for Instance Segmentation, Liu et al. 2018

Arhitectura YOLO – You only look once

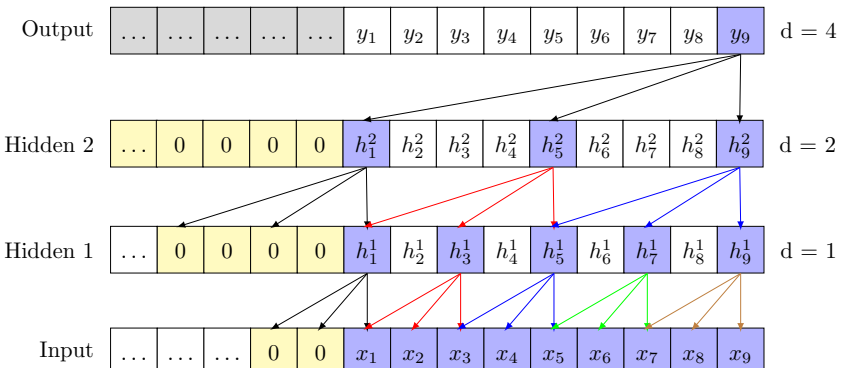
- Multe pre-procesări ale datelor și adaptări ale procesului de antrenare
  - Augmentare de date: augmentare mozaic –> ajută la detecția obiectelor de rezoluție mică într-o imagine
  - Regularizare prin dropout de tip
  - Funcție de activare MISH
  - Funcție de eroare: CioU (Complete intersection over union) – eroare proporțională cu suprafața, distanța și aspectul intersecției de bounding-box-uri ale detecției





$$\mathcal{L}_{CIoU} = 1 - IoU + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v.$$

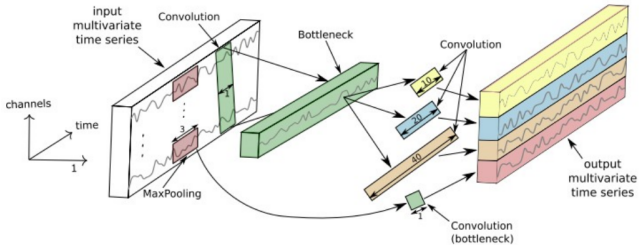
- **Temporal Convolutional Networks** – arhitecturi pentru prelucrarea datelor de tip secvență (e.g. sunet, senzori accelerometru)

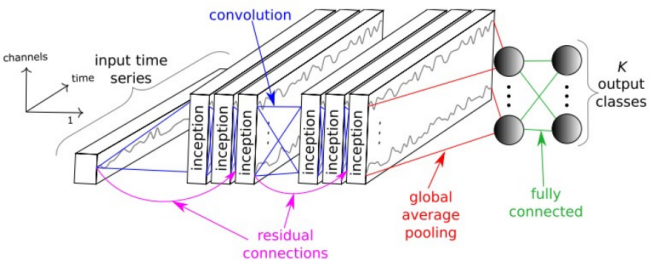


InceptionTime: Finding AlexNet for Time Series Classification", Fawaz et al. 2019

- Arhitectură folosită pentru clasificarea seriilor de timp
- Fructifică ideea de **perspective multiple** asupra aceleiași serii
- Utilizează metode de ansamblu: antrenarea a 5 rețele identice ca arhitectură, dar diferite ca inițializare a ponderilor –> media predicțiilor

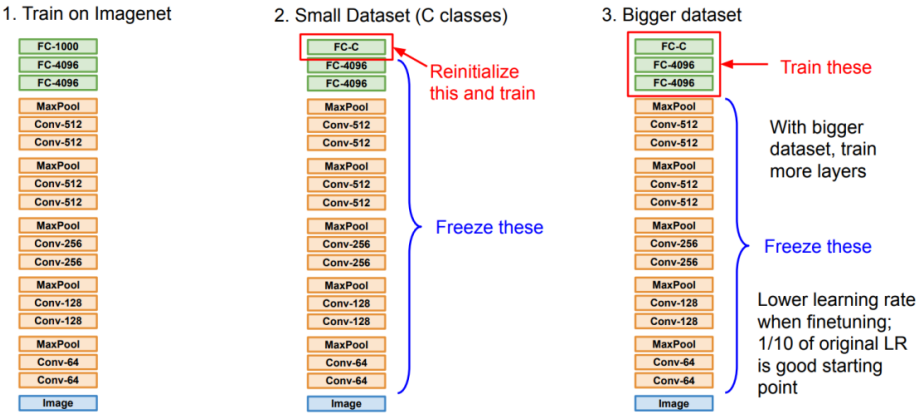
Rețele neurale convoluționale : Arhitecturi și Exemple





Rețele neurale convoluționale : Arhitecturi și Exemple

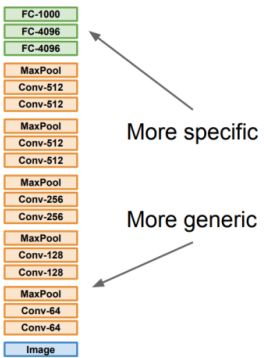
Învățare prin transfer – **procedura de facto** de utilizare a rețelelor convoluțiunile în probleme noi



Sursa: Stanford CS231n Lecture 9, 2019

Rețele neurale convoluționale : Arhitecturi și Exemple

Învățare prin transfer – **procedura de facto** de utilizare a rețelelor convoluțiunile în probleme noi



	very similar dataset	very different dataset
very little data	Use Linear Classifier on top layer	You're in trouble… Try linear classifier from different stages
quite a lot of data	Finetune a few layers	Finetune a larger number of layers

Sursa: Stanford CS231n Lecture 9, 2019

Mulțumesc!



Feedbackul este binevenit!